# FEAT-ZD-48600 · DE-01 Checking the coding prerequisites ■ · Repository (and Databricks) availability and write access

Data Engineering agent · item task.repo_access · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.repo_access.md · approved by Data engineer sim.data_engineer

## task.repo_access: FEAT-ZD-48600

Coding prerequisites for the Data Engineering lane of FEAT-ZD-48600 ("Show which overdue invoices belong to vendors that usually pay late"). This is DE-01, Checking the coding prerequisites (attempt 4). The step checks access to Databricks, that the repository is available, and that the agent identity can write to it. There is no gate: automated checks, the independent judge and sampled human evaluation review it.

| Field | Value | Source |
|---|---|---|
| Trace ID | FEAT-ZD-48600 | record:brief |
| Step | DE-01 Checking the coding prerequisites (attempt 4) | record:brief |
| Trigger | Task creation (feat.tasks from DES-04) | record:brief; feat.tasks |
| Inputs | feat.tasks (b0fd4d2), org.engineering_stack (a6b3e2a) | record:brief |
| Data Engineering tasks in scope | TSK-01 (STORY-01, `paid_date`), TSK-02 (STORY-02, `delays_by_vendor`) | feat.tasks (section 1) |
| Result | **Ready.** The repository is available with write access. Databricks is not applicable. | tool:repo-access; org.engineering_stack |

### 1. Prerequisite checks

| # | Check | Result | Source |
|---|---|---|---|
| P-1 | Databricks workspace access | **Not applicable.** No repository uses Databricks or a database. The tool view says no Databricks or other workspace is used by ap_pipeline. | org.engineering_stack; tool:repo-access |
| P-2 | Repository available | **Pass.** ap_pipeline (Python 3.9, standard library) is available locally. | tool:repo-access; org.engineering_stack |
| P-3 | Repository creation needed | **No.** The repository exists. TSK-01 and TSK-02 are both on the "Data Engineering / ap_pipeline" lane, so no new repository is needed. | feat.tasks (section 1); tool:repo-access |
| P-4 | Integration branches | **Pass.** The repositories have branches `main` (released) and `dev` (integration), with feature branches per story. | org.engineering_stack; feat.tasks (section 2) |
| P-5 | Feature branch created | **Pass.** Branch `feature/story` was created from `dev` at `3fcbe1b`. | tool:repo-access |
| P-6 | Write access for the agent identity | **Pass.** The agent identity has write access and the git status is clean. | tool:repo-access |

| # | Check | Result | Source |
|---|---|---|---|
| P-7 | CI command known | **Pass.** `/usr/bin/python3 ci.py`, which enforces line coverage >= 90% and static checks standing in for SonarQube. The task CI bar adds Python 3.9 syntax only, no `print`, `eval`, `exec` or bare `except`, and lines at most 140 characters. | org.engineering_stack; feat.tasks (section 2) |
| P-8 | Starting version recorded | **Pass.** ap_pipeline is at `0.4.0` today. TSK-01 owns its version step. | feat.tasks (section 2) |
| P-9 | Files in the task budgets | **Recorded.** TSK-01 may change `src/ap_pipeline/transform.py`, `src/ap_pipeline/__init__.py` (version step only) and `tests/test_pipeline.py`. TSK-02 adds the new files `src/ap_pipeline/delays.py` and `tests/test_delays.py`. | feat.tasks (TSK-01, TSK-02) |

## 2. What this means for the next steps

- Secrets: the repository uses no Databricks workspace and no database, so P-1 found no workspace credential to check (org.engineering_stack; tool:repo-access).
- Database changes: no repository uses a database (org.engineering_stack).
- Commits: work goes on `feature/story`. Commit messages start with the story ID (`STORY-01`, `STORY-02`), and the task ID may follow (org.engineering_stack; feat.tasks section 2).
- Scheduling: `schedule.json` is on TSK-01's "Must not change" list, so coding needs no scheduling workspace (feat.tasks TSK-01).

## 3. Observations (non-blocking)

| ID | Observation | Interim handling | Source |
|---|---|---|---|
| RA-1 | The feature branch is named `feature/story`. The convention is `feature/<story>` (for example `feature/STORY-01`). The Data Engineering lane has two stories (STORY-01, STORY-02) on one repository. | Use `feature/story` for both tasks. The story-ID commit prefix keeps each commit traceable. | tool:repo-access; feat.tasks (section 2) |
| RA-2 | Write access is as reported by the tool view. This output does not claim any push or CI run of its own. | None needed for DE-01. | tool:repo-access |

## 4. Validation check

| Check | Result |
|---|---|
| Databricks access checked | Not applicable (P-1) |
| Repository available, no creation needed | Pass (P-2, P-3) |
| Write access | Pass (P-6) |
| Production data read | None |

| Check | Result |
| --- | --- |
| Blocking questions | None |

[Tool result] [ap_pipeline] Branch feature/story at 3fcbe1b; write access confirmed